

Allegro eFlesh Instruction

QtPy SAMD Firmware Setup (mux-binary.cpp)

This document details the firmware configuration and streaming protocol used by `5X-mux-binary-4s/src/mux-binary.cpp` for the Allegro eFlesh tactile sensor system.

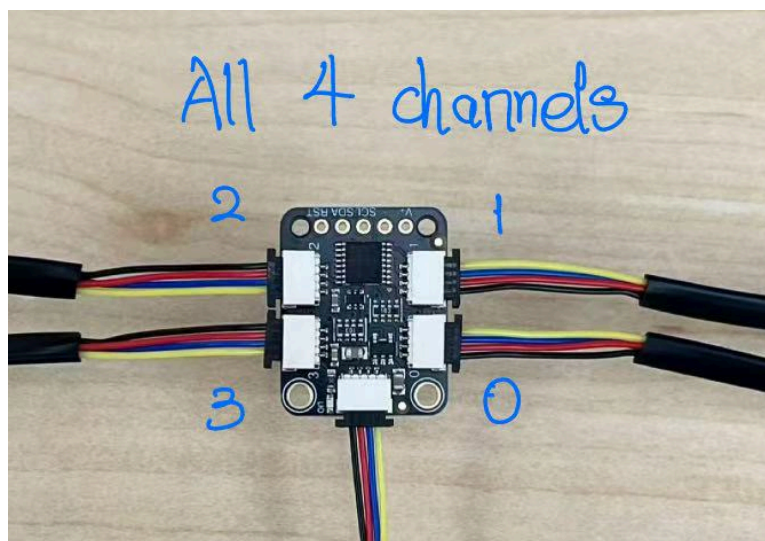
Overview

The `5X-mux-binary-4s/src/mux-binary.cpp` file contains the Arduino firmware that runs on the QtPy SAMD board. This firmware reads from 4 magnetometer branches (via PCA9546A I2C multiplexer) and streams binary tactile data over USB serial.

Hardware Configuration

I2C Multiplexer

- **Type:** PCA9546A (4-channel I2C multiplexer)
- **I2C Address:** `0x70` (default)
- **Channels Used:** 0, 1, 2, 3 (all 4 channels)



Sensor Configuration

- **Sensors per branch:** 5 MLX90393 magnetometers

- **Total sensors:** 4 branches × 5 sensors = 20 sensors
 - **I2C Addresses (white chip):** {0x0C, 0x10, 0x11, 0x12, 0x13} (Notice: This is the chip shipped to you)
 - **I2C Addresses (black chip):** {0x0C, 0x0D, 0x0E, 0x0F, 0x10} (commented out in code)
-

Serial Communication

Baud Rate

```
Serial.begin(230400);
```

Default baud rate: 230400

This is higher than the typical 115200 to support high-frequency streaming. Ensure your Python visualization script matches this baud rate:

```
python allegro_visualizer.py --port /dev/cu.usbmodem1101 --baudrate 230400
```

I2C Clock Speed

```
Wire.setClock(400000);
```

I2C clock: 400 kHz (fast mode)

This allows rapid sensor polling while maintaining stability.

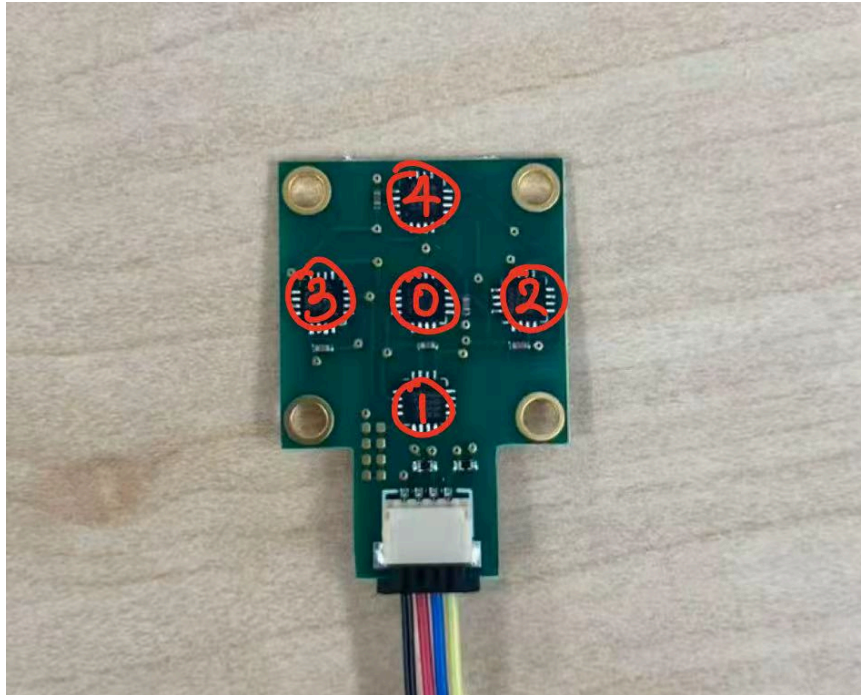
Data Streaming Protocol

Frame Structure

Each frame follows this sequence:

1. **Scan all 4 MUX channels** (branch 0 → branch 3)
2. **For each branch:**
 - Switch to MUX channel via `pcasselect(branch_index)`

- Read 5 sensors sequentially (sequence of 5 sensors)



- For each sensor: write binary `txyz` struct ($4 \times \text{float32} = 16$ bytes)

3. **Frame delimiter:** Send newline (`\n` or `\r\n`)

Binary Payload Format

```
MLX90393::txyz data = {0,0,0,0}; // Temperature, X, Y, Z
Serial.write((byte*)&data, sizeof(data)); // Writes 16 bytes per sensor
```

Per-sensor data:

- `Temperature` (float32) - typically unused for tactile sensing
- `X` (float32) - magnetic field in X-axis
- `Y` (float32) - magnetic field in Y-axis
- `Z` (float32) - magnetic field in Z-axis

Total frame size:

- $20 \text{ sensors} \times 4 \text{ values} \times 4 \text{ bytes} = \mathbf{320 \text{ bytes}}$ per frame
- Plus newline delimiter (1-2 bytes)

Frame Sequencing

```
[newline]
[sensor_0_branch_0: T, X, Y, Z] // 16 bytes
[sensor_1_branch_0: T, X, Y, Z] // 16 bytes
[sensor_2_branch_0: T, X, Y, Z] // 16 bytes
[sensor_3_branch_0: T, X, Y, Z] // 16 bytes
[sensor_4_branch_0: T, X, Y, Z] // 16 bytes
[sensor_0_branch_1: T, X, Y, Z] // 16 bytes
...
[sensor_4_branch_3: T, X, Y, Z] // 16 bytes
[newline]
```

Sensor indexing:

- `branch_index = sensor_id / 5`
- `sensor_in_branch = sensor_id % 5`

Firmware Initialization

Setup Sequence

1. Serial initialization:

```
Serial.begin(230400);
```

2. I2C initialization:

```
Wire.begin();
Wire.setClock(400000);
delay(10);
```

3. Per-branch sensor initialization:

```
for(int idx=0; idx<SENSOR_COUNT; idx++) {
    pcselect(sensor_mux_pins[idx]); // Switch MUX channel
    delay(5); // Allow MUX to settle

    for(int j=0; j<5; j++) {
        mlx[j].begin(mlx_i2c[j], -1, Wire); // Initialize sensor
    }
}
```

```

        mlx[j].setGainSel(1);           // Set gain
        mlx[j].setResolution(1,1,0);    // Set resolution
        mlx[j].startBurst(0xF);         // Start burst mode
    }
}

```

Sensor Configuration

- **Gain selection:** `setGainSel(1)` - Medium gain setting
- **Resolution:** `setResolution(1,1,0)` - X/Y/Z resolution settings
- **Burst mode:** `startBurst(0xF)` - Continuous reading mode (all axes enabled)

Main Loop (Streaming Logic)

```

void loop() {
    for(int idx=0; idx < SENSOR_COUNT; idx++) { // 4 branches
        pcaselect(sensor_mux_pins[idx]);        // Switch MUX channel

        for(int i=0; i<5; i++) {                 // 5 sensors per branch
            mlx[i].readBurstData(data);          // Read T, X, Y, Z
            Serial.write((byte*)&data, sizeof(data)); // Send binary data
        }
    }
    Serial.println(); // Frame delimiter
}

```

Key points:

- No delays between sensor reads (maximizes frame rate)
- MUX switching happens at branch level (not per sensor)
- Binary transmission is more efficient than text formatting
- Newline sent after all 20 sensors are read

Expected Performance

Theoretical Frame Rate

Given:

- **Baud rate:** 230400 bits/second
- **Frame size:** 320 bytes (2560 bits) + ~16 bits overhead = ~2576 bits
- **Effective throughput:** ~80% of baud rate (due to protocol overhead)

Maximum frame rate: $(230400 \times 0.8) / 2576 \approx \mathbf{71.5 \text{ FPS}}$

Practical Considerations

- **I2C read time:** Each MLX90393 burst read takes ~2-3 ms
- **Total I2C time:** 20 sensors $\times$ 2.5 ms $\approx$ 50 ms $\rightarrow$ **~20 FPS theoretical**
- **Actual rate:** Usually 15-30 FPS depending on I2C bus conditions

To increase frame rate:

1. Increase baud rate (e.g., 460800 or 921600)
2. Optimize I2C clock speed (test up to 800 kHz if sensors support it)
3. Reduce sensor count or use faster sensor modes

Python Client Compatibility

The Python visualizer (`allegro_visualizer.py`) expects:

1. **Newline delimiter** before each frame's binary payload
2. **80 float32 values** (little-endian): 20 sensors $\times$ (T, X, Y, Z)
3. **Matching baud rate:** Must match `Serial.begin()` value

Example of visualization from `allegro_visualizer.py` :

```
=====
Allegro eFlesh tactile stream
4 magnetometers x 5 sensors (XYZ)
port=/dev/cu.usbmodem1101 baud=230400
Timestamp: 2025-11-19 13:34:06
FPS: 113.7 | Frames: 7088
=====

Magnetometer Branch 0
Sensor 0 | ΔX -21.29 ΔY -2.21 ΔZ -13.22 | ||Δ||= 25.16
Sensor 1 | ΔX 5.74 ΔY 20.93 ΔZ -1.05 | ||Δ||= 21.73
Sensor 2 | ΔX -16.80 ΔY 5.95 ΔZ -8.58 | ||Δ||= 19.78
Sensor 3 | ΔX 5.62 ΔY -31.75 ΔZ -24.34 | ||Δ||= 40.40
Sensor 4 | ΔX 13.97 ΔY 6.07 ΔZ -9.91 | ||Δ||= 18.17

Magnetometer Branch 1
Sensor 0 | ΔX 2.76 ΔY 15.22 ΔZ -9.12 | ||Δ||= 17.95
Sensor 1 | ΔX 9.89 ΔY 4.90 ΔZ -10.34 | ||Δ||= 15.12
Sensor 2 | ΔX 0.96 ΔY -6.77 ΔZ -18.53 | ||Δ||= 19.75
Sensor 3 | ΔX -9.77 ΔY 6.86 ΔZ -19.21 | ||Δ||= 22.61
Sensor 4 | ΔX -11.81 ΔY -13.99 ΔZ -11.48 | ||Δ||= 21.61

Magnetometer Branch 2
Sensor 0 | ΔX 1.61 ΔY 16.58 ΔZ -8.92 | ||Δ||= 18.90
Sensor 1 | ΔX 8.14 ΔY -7.70 ΔZ -12.53 | ||Δ||= 16.81
Sensor 2 | ΔX 6.82 ΔY 11.35 ΔZ -21.84 | ||Δ||= 25.54
Sensor 3 | ΔX -8.14 ΔY 3.31 ΔZ -18.57 | ||Δ||= 20.54
Sensor 4 | ΔX 0.58 ΔY -7.54 ΔZ -18.55 | ||Δ||= 20.03

Magnetometer Branch 3
Sensor 0 | ΔX 9.22 ΔY -47.28 ΔZ 13.46 | ||Δ||= 50.01
Sensor 1 | ΔX -47.14 ΔY -14.14 ΔZ 8.52 | ||Δ||= 49.94
Sensor 2 | ΔX 13.58 ΔY -36.05 ΔZ 20.81 | ||Δ||= 43.78
Sensor 3 | ΔX 50.23 ΔY 12.14 ΔZ 17.70 | ||Δ||= 54.62
Sensor 4 | ΔX -7.58 ΔY 43.85 ΔZ 11.75 | ||Δ||= 46.02

=====
Total norm (XYZ): 139.67
Average sensor norm: 28.42
Max sensor norm: 54.62
Press 'b' + Enter to reset baseline • Ctrl+C to exit
^C
Stopping stream ...
Goodbye
```

Reading code snippet:

```
serial.read_until(b'\\n') # Skip to frame start
payload = serial.read(320) # Read 80 x float32
floats = struct.unpack('<80f', payload) # Unpack little-endian
```

Code Reference

See [allegro/mux-binary.cpp](#) for the complete firmware source code. Key sections:

- **Lines 27-35:** MUX channel selection function
- **Line 42:** Baud rate configuration
- **Line 45:** I2C clock speed
- **Lines 51-64:** Sensor initialization loop
- **Lines 67-79:** Main streaming loop

For questions or custom modifications, refer to the MLX90393 library documentation and PCA9546A datasheet.

PlatformIO Setup with VS Code (Optional)

To update any microcontroller script to QT Py SAMD, you need instructions below.

1. Install VS Code

Download and install [Visual Studio Code](#) for your operating system.

2. Install PlatformIO Extension

1. Open VS Code
2. Go to Extensions (Ctrl+Shift+X or Cmd+Shift+X)
3. Search for "PlatformIO IDE"
4. Install the extension by PlatformIO
5. Restart VS Code when prompted

3. PlatformIO IDE Setup

The PlatformIO IDE extension will automatically:

- Install PlatformIO Core
- Set up the development environment
- Install necessary toolchains

For detailed setup instructions, visit the [official PlatformIO IDE documentation](#).

4. Open Project

1. In the PlatformIO plugin, Open PIO Home and then click [Open Project](#).
2. Navigate to one of the script directories (e.g., w [wire-scripts/5X-single-board-binary](#))
3. Click "Select Folder"

5. Build and Upload

1. **Build:** Click the PlatformIO icon in the sidebar, then click "Build" (✓ icon)
2. **Upload:** Connect your ESP32 board via USB, then click "Upload" (→ icon)
3. **Monitor:** Click "Monitor" to view serial output (optional)

6. Troubleshooting

- **Port not found:** Ensure your board is connected and drivers are installed
- **Upload fails:** Try pressing the boot button on the QT Py SAMD21 during upload
- **Build errors:** Check that all dependencies are installed (PlatformIO handles this automatically)